Practical Work No. 4: Backup and Restore in SQL Server

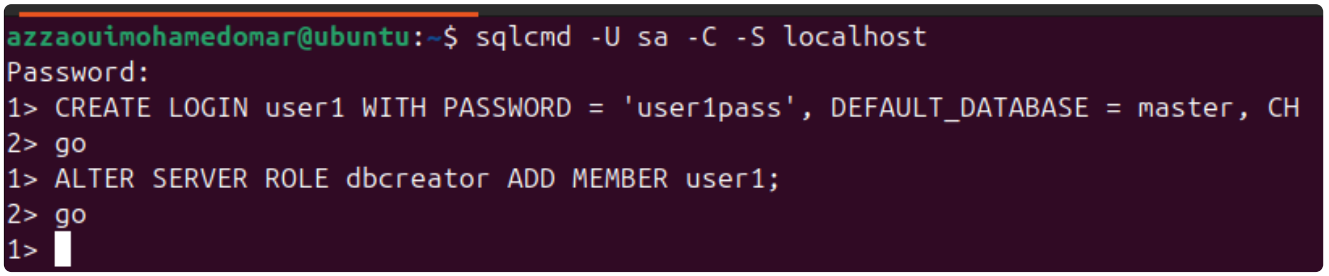
Name: Azzaoui Mohamed Omar
Program: Master Security IT & Big Data
Course: Database Administration
Professor: Prof. C. EL AMRANI

Exercise 1: Full backup and log backup with a logical device

For this exercise we worked on SQL Server 2025 running on Ubuntu, using sqlcmd and DBeaver as clients. Since the server runs on Linux, the Windows paths of the exercise statements (C:\tests) were replaced by /var/opt/mssql/tests/.

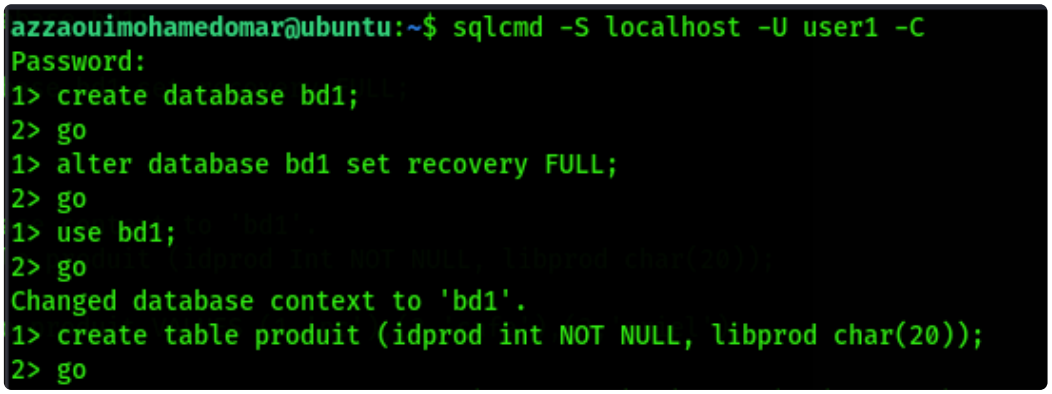
1. Creating the login user1

As sa (super admin), we created the login user1 and gave it the server role dbcreator, which allows it to create databases.



2. Creating the database BD1 and the table produit

Connected as user1, we create the database bd1 and switched it to the FULL recovery model. This choice matters because only the FULL model allows transaction log backups, which we need later to restore the database up to the last saved moment. We then created the table `produit` and inserted three records.



```
azzaouimohamedomar@ubuntu:~$ sqlcmd -S localhost -U user1 -C
Password:
1> use bd1;
2> go
Changed database context to 'bd1'.
1> INSERT INTO produit VALUES (1, 'ariel'), (2, 'megaphone'), (3, 'telephone');
2> go

(3 rows affected)
```

3. Creating the login user2 and giving it its roles

As sa, we create the login user2 and give it the server roles dbcreator, needed to restore databases, and diskadmin, needed to manage backup devices.

```
azzaouimohamedomar@ubuntu:~$ sqlcmd -S localhost -U sa -C
Password:
1>  WITH PASSWORD = 'user2pass', DEFAULT_DATABASE = master, CHECK_POLICY = OFF;
2> go
1> alter server role dbcreator ADD MEMBER user2;
2> go
1> alter SERVER role diskadmin ADD MEMBER user;
2> go
Msg 156, Level 15, State 1, Server ubuntu, Line 1
Incorrect syntax near the keyword 'user'.
1> alter SERVER role diskadmin ADD MEMBER user2;
2> go
```

Then, inside bd1, we declared user2 as a database user and added it to the role db_backupoperator, which gives it the right to back up this database.

```
azzaouimohamedomar@ubuntu:~$ sqlcmd -S localhost -U sa -C
Password:
1> USE bd1;
2> go
Changed database context to 'bd1'.
1> create user user2 for login user2;
2> go
1> alter role db_backupoperator add member user2;
2> go
1>
```

4. Creating the logical backup device Backup1

Connected as user2, we create a logical backup device named Backup1 with sp_addumpdevice. A logical device is simply a name that points to a physical file, so later we can only write TO Backup1 instead of the full path. We then check that it was indeed created by querying the view sys.backup_devices.

```
azzaouimohamedomar@ubuntu:~$ sqlcmd -S localhost -U user2 -C
Password:
1> exec sp_addumpdevice 'disk', 'Backup1', '/var/opt/mssql/tests/SauverBD.bak';
2> go
1> select name,physical_name From sys.backup_devices;
2> go
name
                                                physical_name
------------------------------------------------------------------------------------------------------------------------------------
------------------------------------ ------------------------------------------------------------------------------------------------
------------------------------------------------------------------------------------------------------------------------------------
------------------------------------------------------------------------------------------------------------------------------------
----------------------------------------
Backup1
                                                /var/opt/mssql/tests/SauverBD.bak
```

5. Full backup of BD1

Still as user2, we made a full backup of bd1 on the device Backup1.

```
azzaouimohamedomar@ubuntu:~$ sqlcmd -S localhost -U user2 -C
Password:
1> backup database bd1 TO Backup1 WITH init;
2> go
Processed 416 pages for database 'bd1', file 'bd1' on file 1.
Processed 1 pages for database 'bd1', file 'bd1_log' on file 1.
BACKUP DATABASE successfully processed 417 pages in 0.201 seconds (16.176 MB/sec).
1>
```

6. Adding more records

As user1, we added more products. These rows were inserted after the full backup, so they are not contained in it. Only the log backup will be able to bring them back.

```
azzaouimohamedomar@ubuntu:~$ sqlcmd -S localhost -U user1 -C
Password:
1> INSERT INTO bd1.dbo.produit VALUES (4, 'plush'), (5, 'spoon'), (6, 'martir');
2> go

(3 rows affected)
1>
```

7. Backing up the transaction log

As user2, we back up the transaction log directly into the file BD1_LOG1.trn.

```
azzaouimohamedomar@ubuntu:~$ sqlcmd -S localhost -U user2 -C
Password:
1> backup log bd1 to disk = '/var/opt/mssql/tests/BD1_LOG1.trn';
2> go
Processed 1 pages for database 'bd1', file 'bd1_log' on file 1.
BACKUP LOG successfully processed 1 pages in 0.022 seconds (0.310 MB/sec).
1>
```

Finally, we list the backup folder to confirm that both files were really written to disk.

```
azzaouimohamedomar@ubuntu:~$ sudo ls -l /var/opt/mssql/tests/
[sudo] password for azzaouimohamedomar:
total 3504
-rw-rw---- 1 mssql mssql   84480 Oct 11 15:46 BD1_LOG1.trn
-rw-rw---- 1 mssql mssql 3496448 Oct 11 15:43 SauverBD.bak
```

8. Simulating a disk failure

As sa, we put bd1 offline. Then, in the Ubuntu terminal, we deleted the two files of the database, BD1.mdf and BD1_log.ldf, to simulate a disk crash. From now on the database can only be brought back from our backups.

```
root@ubuntu:/home/azzaouimohamedomar# sudo ls -l /var/opt/mssql/data/ | grep "bd1*"
-rw-rw---- 1 mssql mssql  8388608 Oct 11 15:48 bd1_log.ldf
-rw-rw---- 1 mssql mssql  8388608 Oct 11 15:48 bd1.mdf
-rw-rw---- 1 mssql mssql 14680064 Oct 12 09:20 model_msdbdata.mdf
-rw-rw---- 1 mssql mssql 16187392 Oct 12 09:28 msdbdata.mdf
root@ubuntu:/home/azzaouimohamedomar# /var/opt/mssql/data/
bash: /var/opt/mssql/data/: Is a directory
root@ubuntu:/home/azzaouimohamedomar# cd /var/opt/mssql/data/
root@ubuntu:/var/opt/mssql/data# rm bd1_log.ldf bd1.mdf
root@ubuntu:/var/opt/mssql/data# ls -l /var/opt/mssql/data/ | grep "bd1*"
-rw-rw---- 1 mssql mssql 14680064 Oct 12 09:20 model_msdbdata.mdf
-rw-rw---- 1 mssql mssql 16187392 Oct 12 09:28 msdbdata.mdf
```

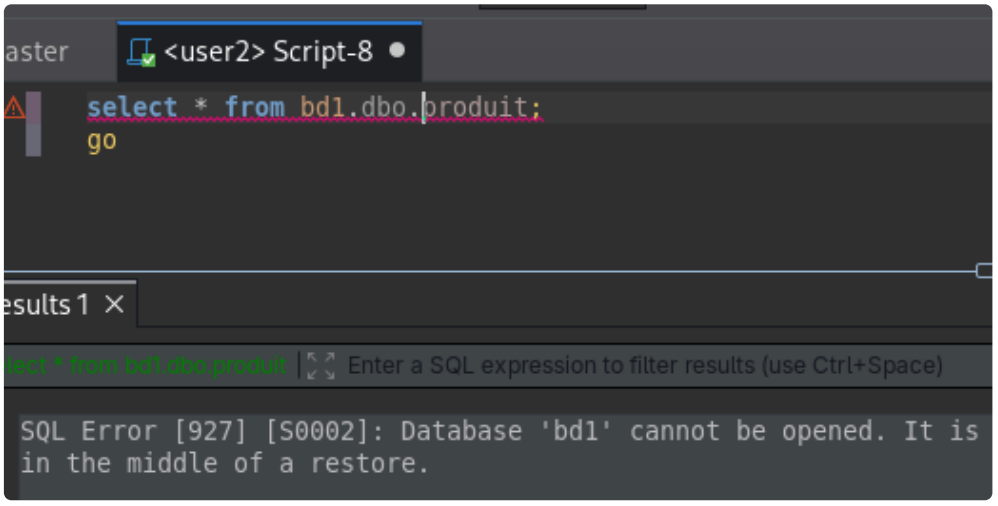
```
azzaouimohamedomar@ubuntu:~$ sqlcmd -S localhost -U sa -C
Password:
1> alter database BD1 SET offline with ROLLBACK immediate;
2> go
```

9. Restoring the full backup with NORECOVERY

As user2, we restored the full backup of bd1 from the device Backup1 with the option NORECOVERY. This option tells SQL Server that more backups are coming, so the restore is not finished yet.

```
azzaouimohamedomar@ubuntu:~$ sqlcmd -S localhost -U user2 -C
Password:
1> restore database bd1 from Backup1 with NORECOVERY,replace;
2> go
Processed 416 pages for database 'bd1', file 'bd1' on file 1.
Processed 1 pages for database 'bd1', file 'bd1_log' on file 1.
RESTORE DATABASE successfully processed 417 pages in 0.050 seconds (65.029 MB/sec).
1> select name,state_desc from sys.databases where name = 'bd1';
2> go
name                                                             state_desc
---------------------------------------------------------------- ----------
bd1                                                              RESTORING

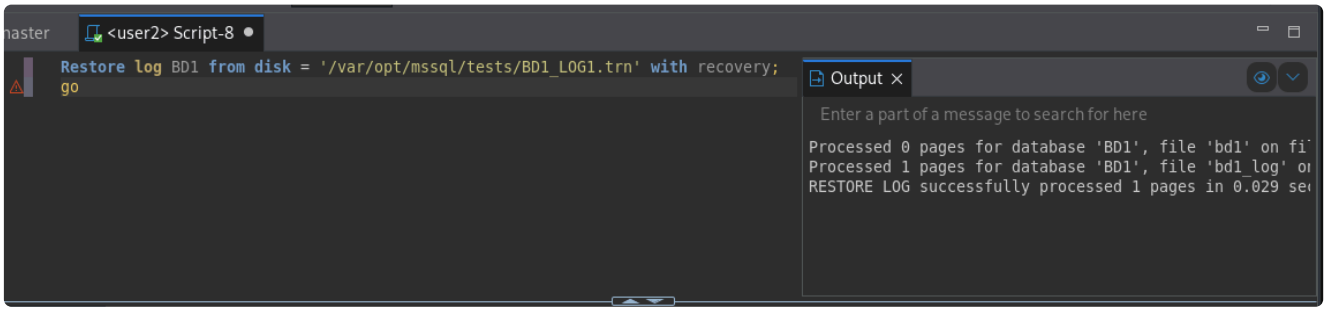
(1 rows affected)
```



Answer: you can't query the table. In DBeaver the database shows that it's in the middle of a restore.

10. Restoring the transaction log with RECOVERY

As user2, we restored the log backup BD1_LOG1.trn with the option RECOVERY. This time the restore is finished, so SQL Server rolls back any unfinished transaction and brings the database back online. We then displayed the table produit.



```
azzaouimohamedomar@ubuntu:~$ sqlcmd -S localhost -U user1 -C
Password:
1> SELECT * from bd1.dbo.produit;
2> go
idprod      libprod
----------- --------------------
          1 ariel
          2 megaphone
          3 telephone
          4 plush
          5 spoon
          6 martir
(6 rows affected)
```

The table contains all the products again, including the ones added after the full backup (products 4 to 6). They were recovered thanks to the log backup, which shows that the full backup and the log backup together bring the database back to its last saved state.

Exercise 2: Full, differential and log backups

Setup: the database and the 10 clients

As sa, we created the database Gestion in the FULL recovery model, with the table clients, and inserted 10 clients.

```
azzaouimohamedomar@ubuntu:~$ sqlcmd -U sa -S localhost -C
Password:
1> use Gestion
2> go
Changed database context to 'Gestion'.
1> Alter database Gestion set recovery FULL;
2> go
1> create table clients(idclient int primary key,nom varchar(50), ville varchar(50));
2> go
1> insert into clients values (1,'Amine','Casablanca'),(2,'Sara','Rabat'),(3,'Youssef','F
es'),(4,'Salma','Tanger'),(5,'Omar','Agadir'),(6,'Hiba','Marrakech'),(7,'Karim','Oujda'),
(8,'Nada','Meknes'),(9,'Reda','Tetouan'),(10,'im
2> go

(10 rows affected)
```

We then checked that the table really contains 10 rows.

```
1> select count(*) from clients;
2> go

-----------
         10

(1 rows affected)
```

1. Full backup

We made a full backup of Gestion in the file Gestion_FULL.bak.

```
azzaouimohamedomar@ubuntu:~$ sqlcmd -U sa -S localhost -C
Password:
1> BACKUP database Gestion to disk='/var/opt/mssql/tests/Gestion_FULL.bak' with init;
2> go
Processed 416 pages for database 'Gestion', file 'Gestion' on file 1.
Processed 1 pages for database 'Gestion', file 'Gestion_log' on file 1.
BACKUP DATABASE successfully processed 417 pages in 0.221 seconds (14.712 MB/sec).
```

2. Three clients, then a log backup

We inserted 3 clients, then backed up the transaction log in Gestion_LOG1.trn.

```
azzaouimohamedomar@ubuntu:~$ sqlcmd -U sa -S localhost -C
Password:
1> use Gestion;
2> go
Changed database context to 'Gestion'.
1> insert into clients values (11,'Zineb','Safi'),(12,'Moussa','Tantan'),
2> (13,'Laarbi','Sidi Rahal');
3> go

(3 rows affected)
1> backup log Gestion to disk = '/var/opt/mssql/tests/Gestion_LOG1.trn';
2> go
Processed 1 pages for database 'Gestion', file 'Gestion_log' on file 1.
BACKUP LOG successfully processed 1 pages in 0.013 seconds (0.525 MB/sec).
1>
```

3. Three clients, then a differential backup

We inserted 3 more clients, then made a differential backup in Gestion_DIFF.bak. It contains everything that changed since the full backup.

```
azzaouimohamedomar@ubuntu:~$ sqlcmd -U sa -S localhost -C
Password:
1> use Gestion;
2> go
Changed database context to 'Gestion'.
1> Insert into clients values (14,'Mehdi','Settat'),(15,'Rim','Berkane'),(16,'Adam','Taza');
2> go

(3 rows affected)
1> backup database Gestion to disk='/var/opt/mssql/tests/Gestion_DIFF.bak' with differential;
2> go
Processed 104 pages for database 'Gestion', file 'Gestion' on file 1.
Processed 1 pages for database 'Gestion', file 'Gestion_log' on file 1.
BACKUP DATABASE WITH DIFFERENTIAL successfully processed 105 pages in 0.142 seconds (5.732 MB/sec).
1>
```

4. Three clients, then a log backup

We inserted 3 more clients, then backed up the transaction log in Gestion_LOG2.trn.

```
azzaouimohamedomar@ubuntu:~$ sqlcmd -U sa -S localhost -C
Password:
1> use Gestion;
2> go
Changed database context to 'Gestion'.
1> insert into clients values (17,'Salah','Larache'),(18,'Dounia','Azrou'),(19,'Ilyas','Errachidia');
2> go

(3 rows affected)
1> BACKUP log Gestion to disk = '/var/opt/mssql/tests/Gestion_LOG2.trn';
2> go
Processed 3 pages for database 'Gestion', file 'Gestion_log' on file 1.
BACKUP LOG successfully processed 3 pages in 0.014 seconds (1.220 MB/sec).
```

5. Three clients, then a log backup

We inserted the last 3 clients, then backed up the transaction log in Gestion_LOG3.trn.

```
azzaouimohamedomar@ubuntu:~$ sqlcmd -U sa -S localhost -C
Password:
1> use Gestion;
2> go
Changed database context to 'Gestion'.
1> insert into values clients (20,'Chaima','Essaouira'),(21,'Yassine','Beni Mellal'),(22,'Meryem','El
2> go
Msg 156, Level 15, State 1, Server ubuntu, Line 1
Incorrect syntax near the keyword 'values'.
1> backup log Gestion to disk='/var/opt/mssql/tests/Gestion_LOG3.trn';
2> go
Processed 1 pages for database 'Gestion', file 'Gestion_log' on file 1.
BACKUP LOG successfully processed 1 pages in 0.023 seconds (0.127 MB/sec).
```

To be sure everything is in place, we listed the backup folder and checked that all the backup files were written.

```
azzaouimohamedomar@ubuntu:~$ sudo ls -ltr /var/opt/mssql/tests/
[sudo] password for azzaouimohamedomar:
total 8096
-rw-rw---- 1 mssql mssql 3496448 Oct 11 15:43 SauverBD.bak
-rw-rw---- 1 mssql mssql   84480 Oct 11 15:46 BD1_LOG1.trn
-rw-rw---- 1 mssql mssql 3496448 Oct 19 15:49 Gestion_FULL.bak
-rw-rw---- 1 mssql mssql   84480 Oct 19 15:54 Gestion_LOG1.trn
-rw-rw---- 1 mssql mssql  940544 Oct 19 15:56 Gestion_DIFF.bak
-rw-rw---- 1 mssql mssql   84480 Oct 19 15:58 Gestion_LOG2.trn
-rw-rw---- 1 mssql mssql   84480 Oct 19 16:00 Gestion_LOG3.trn
```

6. Which backups to restore

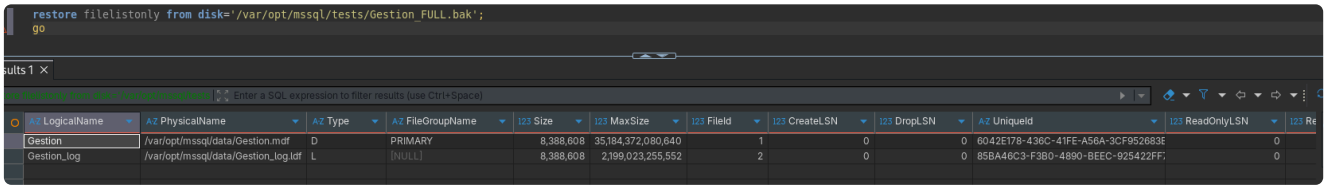
To get the latest state back, we must restore in this order:

1. Gestion_FULL.bak with NORECOVERY
2. Gestion_DIFF.bak with NORECOVERY
3. Gestion_LOG2.trn with NORECOVERY
4. Gestion_LOG3.trn with RECOVERY

Gestion_LOG1.trn is useless because the differential backup contains every change made since the full backup, and that includes everything LOG1 holds. Only the logs taken after the differential are needed.

7. Restoring under another name: Gestion_Copie

We restored the chain into a new database called Gestion_Copie. Since the original database still exists, the option MOVE is needed to give the data file and the log file new names and locations. The logical file names, Gestion and Gestion_log, were read from the backup file.



The restore starts with the full backup, using NORECOVERY:

```
restore database Gestion_Copie from
disk='/var/opt/mssql/tests/Gestion_FULL.bak' with move 'Gestion' to
'/var/opt/mssql/data/Gestion_Copie.mdf', move 'Gestion_log' to
'/var/opt/mssql/data/Gestion_Copie_log.ldf', NORECOVERY;
```

Note that our first count showed only 19 clients: the last 3 clients had not been inserted in Gestion when LOG3 was taken. We inserted them afterwards and made one more log backup, Gestion_LOG4.trn, so the chain we restored is the full backup, the differential, LOG2, LOG3 and LOG4. After that, Gestion_Copie contains the 22 clients.

```
1> pie from disk='/var/opt/mssql/tests/Gestion_FULL.bak' with move 'Gestion' to '/var/opt/mssql/dat
2> go
Processed 416 pages for database 'Gestion_Copie', file 'Gestion' on file 1.
Processed 1 pages for database 'Gestion_Copie', file 'Gestion_log' on file 1.
RESTORE DATABASE successfully processed 417 pages in 0.061 seconds (53.302 MB/sec).
1> RESTORE database Gestion_Copie from disk='/var/opt/mssql/tests/Gestion_DIFF.bak' with norecovery
2> go
Processed 104 pages for database 'Gestion_Copie', file 'Gestion' on file 1.
Processed 1 pages for database 'Gestion_Copie', file 'Gestion_log' on file 1.
RESTORE DATABASE successfully processed 105 pages in 0.058 seconds (14.033 MB/sec).
1> restore log Gestion_Copie from disk='/var/opt/mssql/tests/Gestion_LOG2.trn' with NORECOVERY;
2> go
Processed 0 pages for database 'Gestion_Copie', file 'Gestion' on file 1.
Processed 3 pages for database 'Gestion_Copie', file 'Gestion_log' on file 1.
RESTORE LOG successfully processed 3 pages in 0.035 seconds (0.488 MB/sec).
1> Restore log Gestion_Copie from disk='/var/opt/mssql/tests/Gestion_LOG3.trn' with norecovery;
2> go
Processed 0 pages for database 'Gestion_Copie', file 'Gestion' on file 1.
Processed 1 pages for database 'Gestion_Copie', file 'Gestion_log' on file 1.
RESTORE LOG successfully processed 1 pages in 0.041 seconds (0.071 MB/sec).
1> restore log Gestion_Copie from disk='/var/opt/mssql/tests/Gestion_LOG4.trn' with recovery;
2> go
Processed 0 pages for database 'Gestion_Copie', file 'Gestion' on file 1.
Processed 1 pages for database 'Gestion_Copie', file 'Gestion_log' on file 1.
RESTORE LOG successfully processed 1 pages in 0.036 seconds (0.108 MB/sec).
1> select count(*) as nb from Gestion_Copie.dbo.clients;
2> go
nb
-----------
         22

(1 rows affected)
1>
```

8. Restoring the differential with RECOVERY, then LOG2

To test this, we restored the full backup with NORECOVERY and then the differential with RECOVERY, in a separate test database. The database came online, and when we tried to

restore Gestion_LOG2.trn SQL Server refused it with the error "no files are ready to rollforward".

```
1> select name,state_desc from sys.databases where name = 'Gestion_Test';
2> go
name
                    state_desc
------------------------------------------------------------------------------------------------------------------------
-------------------- ------------------------------------------------------------
Gestion_Test
                    ONLINE

(1 rows affected)
1> restore log Gestion_Test from disk='/var/opt/mssql/tests/Gestion_LOG2.trn' with norecovery;
2> go
Msg 3117, Level 16, State 4, Server ubuntu, Line 1
The log or differential backup cannot be restored because no files are ready to rollforward.
Msg 3013, Level 16, State 1, Server ubuntu, Line 1
RESTORE LOG is terminating abnormally.
1>
```

RECOVERY ends the restore sequence. SQL Server rolls back the unfinished transactions and brings the database online, so it refuses any further backup. To apply LOG2 and the next logs, the differential has to be restored with NORECOVERY, as we did for Gestion_Copie. Otherwise we have to start again from the full backup.

Exercise 3: Point-in-time restore after an error

1. New full backup

We made a new full backup of Gestion in the file Gestion_FULL2.bak.

```
azzaouimohamedomar@ubuntu:~$ sqlcmd -U sa -S localhost -C
Password:
1> backup database Gestion to disk='/var/opt/mssql/tests/Gestion_FULL2.bak' with init;
2> go
Processed 416 pages for database 'Gestion', file 'Gestion' on file 1.
Processed 1 pages for database 'Gestion', file 'Gestion_log' on file 1.
BACKUP DATABASE successfully processed 417 pages in 0.182 seconds (17.865 MB/sec).
```

2. Inserting 2 clients, noting the time, then the mistake

We inserted 2 clients and noted the exact time with SELECT SYSDATETIME(). After waiting one minute, we ran the mistake: DELETE FROM clients, without any WHERE clause. The table is now empty.

```
1> use Gestion;
2> go
Changed database context to 'Gestion'.
1> insert into clients values (23,'Hamza','Ifrane'),(24,'Aya','Midelt');
2> go

(2 rows affected)
1> SELECT sysdatetime();
2> go

------------------------------------
          2026-10-19 16:58:49.5814640

(1 rows affected)
1> delete from clients;
2> go

(24 rows affected)
1> select count(*) from Gestion.dbo.clients;
2> go

-----------
          0

(1 rows affected)
```

3. Transaction log backup

We backed up the transaction log in Gestion_LOG5.trn. This file contains the 2 new clients and also the DELETE.

```
1> Backup log Gestion to disk='/var/opt/mssql/tests/Gestion_LOG5.trn';
2> go
Processed 4 pages for database 'Gestion', file 'Gestion_log' on file 1.
BACKUP LOG successfully processed 4 pages in 0.027 seconds (1.139 MB/sec).
```

4. Restoring just before the DELETE

We restored the full backup under the name Gestion_Restore with the option MOVE and NORECOVERY, then restored the log with the option STOPAT. The stop time must be after the full backup and before the DELETE. We used the time we noted, plus one second. Our first try used an earlier time, which SQL Server refused with the error "STOPAT time is too early", because it was before the full backup.

```
azzaouimohamedomar@ubuntu:~$ sqlcmd -U sa -S localhost -C
Password:
1> RESTORE log Gestion_Restore from disk='/var/opt/mssql/tests/Gestion_LOG5.trn' with stopat='2026-10-19 1
2> go
Processed 0 pages for database 'Gestion_Restore', file 'Gestion' on file 1.
Processed 4 pages for database 'Gestion_Restore', file 'Gestion_log' on file 1.
RESTORE LOG successfully processed 4 pages in 0.056 seconds (0.549 MB/sec).
1> select count(*) as nb from Gestion_Restore.dbo.clients;
2> go
nb
-----------
         24

(1 rows affected)
1>
```

The restored database contains 24 clients: the 22 existing clients and the 2 we inserted just before the mistake.

5. Copying the clients back into Gestion

Since Gestion is empty after the DELETE, we copied the clients back with INSERT INTO ... SELECT ... from Gestion_Restore.

```
1> insert into Gestion.dbo.clients select * from Gestion_Restore.dbo.clients;
2> go

(24 rows affected)
1>
```

6. Difference between NORECOVERY and RECOVERY

NORECOVERY leaves the database in the restoring state, so it can't be queried. SQL Server keeps the unfinished transactions open because the next backup in the chain, a differential or a log, may complete them. RECOVERY finishes the process: SQL Server rolls back the unfinished transactions and brings the database online, and no further backup can be applied afterwards. So we use NORECOVERY on every backup except the last one, and RECOVERY on the last one only.

Exercise 4: Automating the backups of BD1

SQL Server runs on Linux here and there is no graphical interface for SQL Server Agent in DBeaver, so the jobs were created with T-SQL scripts, and the job history was read from the msdb database.

1. Enabling SQL Server Agent and creating the two jobs

First we enabled SQL Server Agent with mssql-conf and restarted the SQL Server service.

```
azzaouimohamedomar@ubuntu:~$ sudo /opt/mssql/bin/mssql-conf set sqlagent.enabled true
[sudo] password for azzaouimohamedomar:
SQL Server needs to be restarted in order to apply this setting. Please run
'systemctl restart mssql-server.service'.
Note that sp_reload_mssqlconf can be used to apply a limited subset of settings without restarting the serv
ice.
azzaouimohamedomar@ubuntu:~$ sudo systemctl restart mssql-server
Warning: The unit file, source configuration file or drop-ins of mssql-server.service changed on disk. Run
'systemctl daemon-reload' to reload units.
azzaouimohamedomar@ubuntu:~$ sqlcmd -U sa -S localhost -C
Password:
1> select name,state_desc,recovery_model_desc from sys.databases where name = 'BD1';
2> go
name
                    state_desc                                                   recovery_model_desc

------------------------------------------------------------------------------------------------------------
-------------------- ------------------------------------------------------------ ----------------------------
------------------------------------
bd1
                    ONLINE                                                       FULL


(1 rows affected)
```

Then we created two jobs with a script. The first one makes a full backup of BD1 every 10 minutes, and the second one backs up the log of BD1 every 2 minutes. The file name is built with

the date and the time, for example BD1_LOG_20261019_1732.trn, so no backup overwrites the previous one.

```
azzaouimohamedomar@ubuntu:~$ sqlcmd -S localhost -U sa -C -i ~/jobs_bd1.sql
Password:
Changed database context to 'msdb'.
azzaouimohamedomar@ubuntu:~$ sqlcmd -S localhost -U sa -C
Password:
1> select name,enabled from msdb.dbo.sysjobs;
2> go
name                                                                                                                             enabled
-------------------------------------------------------------------------------------------------------------------------------- -------
Log_BD1                                                                                                                                1
Full_BD1                                                                                                                               1

(2 rows affected)
1> exec msdb.dbo.sp_start_job N'Full_BD1';
2> go
Job 'Full_BD1' started successfully.
1>
```

We checked that the jobs exist, and that the backup files appear in the backup folder with the date and time in their names.

```
-rw-rw---- 1 mssql mssql 3758592 Oct 19 17:16 BD1_FULL_20261019_1716.bak
azzaouimohamedomar@ubuntu:~$
```

2. Inserting a row every 10 seconds for 15 minutes

Before starting, we counted the rows of the table produit, to know how many rows we should have at the end.

```
azzaouimohamedomar@ubuntu:~$ sqlcmd -S localhost -U sa -C
Password:
1> select count(*) from BD1.dbo.produit;
2> go

-----------
          6

(1 rows affected)
1>
```

Then we ran the script loop_bd1.sql. It is a WHILE loop with WAITFOR DELAY '00:00:10' that inserts 90 rows, one every 10 seconds, which makes 15 minutes.

```
azzaouimohamedomar@ubuntu:~$ cat ~/loop_bd1.sql
set nocount on
declare @i int = 1
while @i <= 90
begin
  insert into BD1.dbo.produit values (100 + @i, 'prod_loop')
  waitfor delay '00:00:10'
  set @i = @i + 1
end
go
azzaouimohamedomar@ubuntu:~$ sqlcmd -S localhost -U user1 -C -i ~/loop_bd1.sql
Password:
Sqlcmd: Error: Microsoft ODBC Driver 18 for SQL Server : Login failed for user 'user1'..
azzaouimohamedomar@ubuntu:~$ sqlcmd -S localhost -U user1 -C -i ~/loop_bd1.sql
Password:
```

After the 15 minutes, we counted the rows again. The table contains the 90 new rows. Then we forced one last log backup, so the last rows are also covered by a backup, and disabled the two jobs.

```
azzaouimohamedomar@ubuntu:~$ sqlcmd -S localhost -U sa -C
Password:
1> select count(*) from BD1.dbo.produit;
2> go

-----------
         96

(1 rows affected)
1>
```

3. Checking the history of the jobs

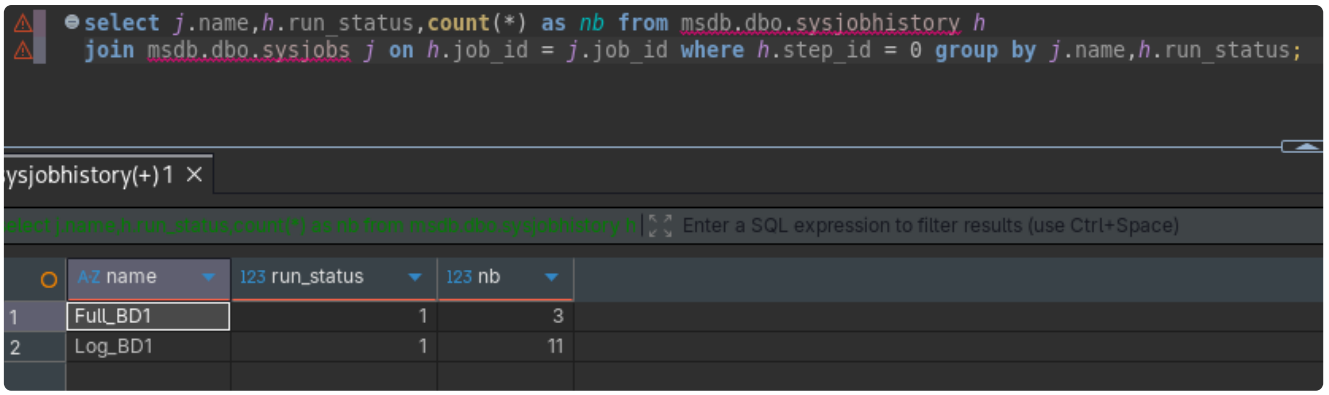
We read the history of the jobs from msdb. A run_status of 1 means that the execution succeeded, and 0 means that it failed. We checked that no execution failed.

```
select j.name, msdb.dbo.agent_datetime(h.run_date,h.run_time) as quand,
  h.run_status, h.message from msdb.dbo.sysjobhistory h join msdb.dbo.sysjobs j on h.job_id = j.job_id where h.step_id = 0 order by h.instance_id desc;
```

ysjobhistory(+)1

Enter a SQL expression to filter results (use Ctrl+Space)

	name	quand	run_status	message
1	Log_BD1	2026-10-19 17:36:28.000	1	The job succeeded. The Job was invoked by User sa. The last step to run was step 1 (sauvegarde_log).
2	Log_BD1	2026-10-19 17:36:00.000	1	The job succeeded. The Job was invoked by Schedule 9 (toutes_les_2_min). The last step to run was step 1 (sauvegarde_log).
3	Log_BD1	2026-10-19 17:34:00.000	1	The job succeeded. The Job was invoked by Schedule 9 (toutes_les_2_min). The last step to run was step 1 (sauvegarde_log).
4	Log_BD1	2026-10-19 17:32:00.000	1	The job succeeded. The Job was invoked by Schedule 9 (toutes_les_2_min). The last step to run was step 1 (sauvegarde_log).
5	Log_BD1	2026-10-19 17:30:00.000	1	The job succeeded. The Job was invoked by Schedule 9 (toutes_les_2_min). The last step to run was step 1 (sauvegarde_log).
6	Full_BD1	2026-10-19 17:30:00.000	1	The job succeeded. The Job was invoked by Schedule 8 (toutes_les_10_min). The last step to run was step 1 (sauvegarde_full).
7	Log_BD1	2026-10-19 17:28:00.000	1	The job succeeded. The Job was invoked by Schedule 9 (toutes_les_2_min). The last step to run was step 1 (sauvegarde_log).
8	Log_BD1	2026-10-19 17:26:00.000	1	The job succeeded. The Job was invoked by Schedule 9 (toutes_les_2_min). The last step to run was step 1 (sauvegarde_log).
9	Log_BD1	2026-10-19 17:24:00.000	1	The job succeeded. The Job was invoked by Schedule 9 (toutes_les_2_min). The last step to run was step 1 (sauvegarde_log).
10	Log_BD1	2026-10-19 17:22:00.000	1	The job succeeded. The Job was invoked by Schedule 9 (toutes_les_2_min). The last step to run was step 1 (sauvegarde_log).
11	Log_BD1	2026-10-19 17:20:01.000	1	The job succeeded. The Job was invoked by Schedule 9 (toutes_les_2_min). The last step to run was step 1 (sauvegarde_log).
12	Full_BD1	2026-10-19 17:20:01.000	1	The job succeeded. The Job was invoked by Schedule 8 (toutes_les_10_min). The last step to run was step 1 (sauvegarde_full).
13	Log_BD1	2026-10-19 17:18:01.000	1	The job succeeded. The Job was invoked by Schedule 9 (toutes_les_2_min). The last step to run was step 1 (sauvegarde_log).
14	Full_BD1	2026-10-19 17:16:19.000	1	The job succeeded. The Job was invoked by User sa. The last step to run was step 1 (sauvegarde_full).



4. Dropping BD1 and restoring it

Before dropping the database, we looked at the backup history. SSMS proposes the most recent full backup plus all the log backups taken after it, in order, because the log backups form a chain that brings the database to the last saved moment. The older fulls and logs aren't needed.

```
backup_start_date       type physical_device_name

----------------------- ---- ------------------------------------------------------------
----------------------------------------------------
2026-10-11 15:43:01.000 D    /var/opt/mssql/tests/SauverBD.bak

2026-10-11 15:46:06.000 L    /var/opt/mssql/tests/BD1_LOG1.trn

2026-10-19 17:16:19.000 D    /var/opt/mssql/tests/BD1_FULL_20261019_1716.bak

2026-10-19 17:18:01.000 L    /var/opt/mssql/tests/BD1_LOG_20261019_1718.trn

2026-10-19 17:20:01.000 D    /var/opt/mssql/tests/BD1_FULL_20261019_1720.bak

2026-10-19 17:20:01.000 L    /var/opt/mssql/tests/BD1_LOG_20261019_1720.trn

2026-10-19 17:22:01.000 L    /var/opt/mssql/tests/BD1_LOG_20261019_1722.trn

2026-10-19 17:24:00.000 L    /var/opt/mssql/tests/BD1_LOG_20261019_1724.trn

2026-10-19 17:26:00.000 L    /var/opt/mssql/tests/BD1_LOG_20261019_1726.trn

2026-10-19 17:28:00.000 L    /var/opt/mssql/tests/BD1_LOG_20261019_1728.trn

2026-10-19 17:30:00.000 D    /var/opt/mssql/tests/BD1_FULL_20261019_1730.bak

2026-10-19 17:30:01.000 L    /var/opt/mssql/tests/BD1_LOG_20261019_1730.trn

2026-10-19 17:32:00.000 L    /var/opt/mssql/tests/BD1_LOG_20261019_1732.trn

2026-10-19 17:34:00.000 L    /var/opt/mssql/tests/BD1_LOG_20261019_1734.trn

2026-10-19 17:36:00.000 L    /var/opt/mssql/tests/BD1_LOG_20261019_1736.trn

2026-10-19 17:36:28.000 L    /var/opt/mssql/tests/BD1_LOG_20261019_1736.trn
```

Then we dropped the database BD1.

```
1> alter database BD1 set single_user with rollback immediate;
2> go
1> drop database BD1;
2> go
1>
```

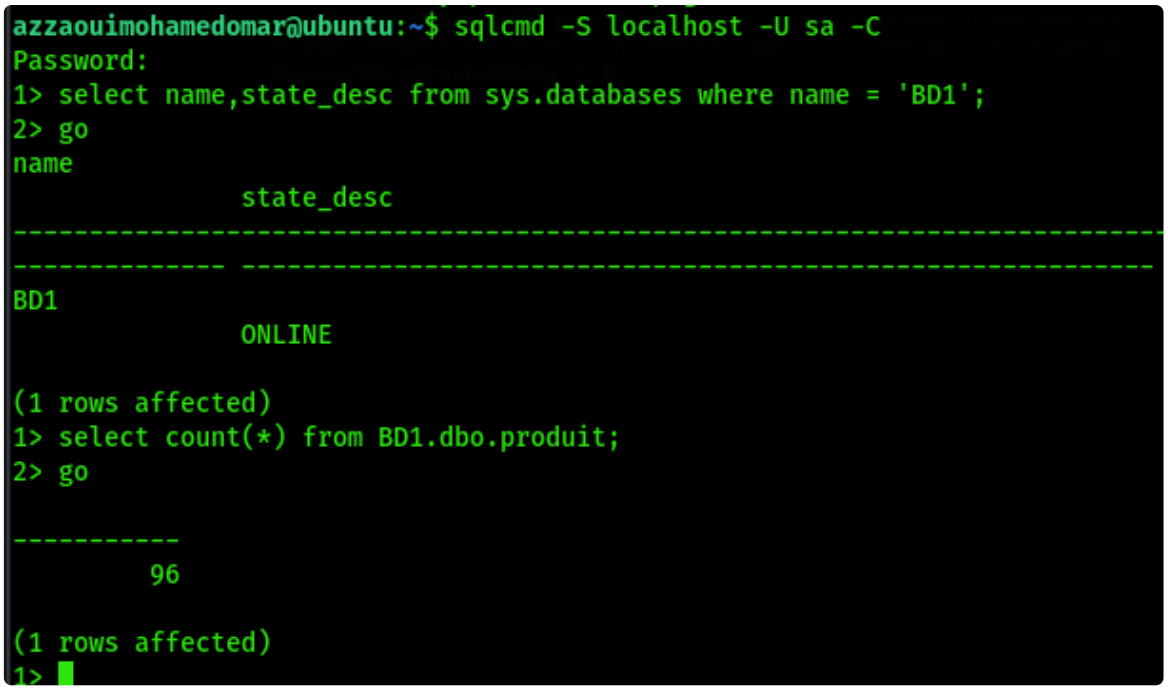
Since SSMS is not available on Linux, we generated the restore commands from the backup history: one RESTORE DATABASE for the last full backup, one RESTORE LOG for each log backup in order, and a final RECOVERY. We then ran the generated file with sqlcmd.

```
azzaouimohamedomar@ubuntu:~$ cat ~/gen_restore.sql


set nocount on
declare @fckpt numeric(25,0), @fmedia int
select top 1 @fckpt = checkpoint_lsn, @fmedia = media_set_id from msdb.dbo.backupset where database_name = 'BD1' a
nd type = 'D' order by backup_start_date desc
select cmd from (
  select 0 as o, null as d, 'restore database BD1 from disk=''' + m.physical_device_name + ''' with norecovery, re
place;' as cmd from msdb.dbo.backupmediafamily m where m.media_set_id = @fmedia
  union all
  select 1, b.backup_start_date, 'restore log BD1 from disk=''' + m.physical_device_name + ''' with norecovery;' f
rom msdb.dbo.backupset b join msdb.dbo.backupmediafamily m on b.media_set_id = m.media_set_id where b.database_nam
e = 'BD1' and b.type = 'L' and b.database_backup_lsn = @fckpt
) x order by o, d
go
azzaouimohamedomar@ubuntu:~$ sqlcmd -S localhost -U sa -P 'f@S2ubs4y4ccX' -C -h -1 -W -i ~/gen_restore.sql -o ~/re
store_bd1.sql

azzaouimohamedomar@ubuntu:~$ echo "restore database BD1 with recovery;" >> ~/restore_bd1.sql
azzaouimohamedomar@ubuntu:~$ cat ~/restore_bd1.sql
restore database BD1 from disk='/var/opt/mssql/tests/BD1_FULL_20261019_1730.bak' with norecovery, replace;
restore log BD1 from disk='/var/opt/mssql/tests/BD1_LOG_20261019_1730.trn' with norecovery;
restore log BD1 from disk='/var/opt/mssql/tests/BD1_LOG_20261019_1732.trn' with norecovery;
restore log BD1 from disk='/var/opt/mssql/tests/BD1_LOG_20261019_1734.trn' with norecovery;
restore log BD1 from disk='/var/opt/mssql/tests/BD1_LOG_20261019_1736.trn' with norecovery;
restore log BD1 from disk='/var/opt/mssql/tests/BD1_LOG_20261019_1736.trn' with norecovery;
restore database BD1 with recovery;
azzaouimohamedomar@ubuntu:~$ sqlcmd -S localhost -U sa -C -i ~/restore_bd1.sql
Password:
Processed 448 pages for database 'BD1', file 'bd1' on file 1.
Processed 1 pages for database 'BD1', file 'bd1_log' on file 1.
RESTORE DATABASE successfully processed 449 pages in 0.061 seconds (57.401 MB/sec).
Processed 0 pages for database 'BD1', file 'bd1' on file 1.
Processed 4 pages for database 'BD1', file 'bd1_log' on file 1.
RESTORE LOG successfully processed 4 pages in 0.022 seconds (1.198 MB/sec).
Processed 0 pages for database 'BD1', file 'bd1' on file 1.
Processed 2 pages for database 'BD1', file 'bd1_log' on file 1.
RESTORE LOG successfully processed 2 pages in 0.032 seconds (0.457 MB/sec).
Processed 0 pages for database 'BD1', file 'bd1' on file 1.
Processed 2 pages for database 'BD1', file 'bd1_log' on file 1.
RESTORE LOG successfully processed 2 pages in 0.025 seconds (0.351 MB/sec).
Processed 0 pages for database 'BD1', file 'bd1' on file 1.
Processed 1 pages for database 'BD1', file 'bd1_log' on file 1.
RESTORE LOG successfully processed 1 pages in 0.023 seconds (0.127 MB/sec).
Processed 0 pages for database 'BD1', file 'bd1' on file 1.
Processed 1 pages for database 'BD1', file 'bd1_log' on file 1.
RESTORE LOG successfully processed 1 pages in 0.034 seconds (0.086 MB/sec).
RESTORE DATABASE successfully processed 0 pages in 0.463 seconds (0.000 MB/sec).
azzaouimohamedomar@ubuntu:~$
```

Finally, we checked that BD1 is back online and counted its rows. The count is the same as the one we noted before the drop, because the final log backup covered the last rows. If the count had been lower, it would mean the last insert came after the last log backup.



Exercise 6: Backup plan for an online bookstore

1. Recovery model

The database will use the FULL recovery model because it supports transaction log backups and point-in-time recovery. Log backups will run every five minutes to keep data loss within the 15-minute RPO. The backup and restore process must be tested regularly to make sure the database can be brought back online in less than one hour.

2. Weekly backup schedule

Day	Full backup	Differential backups	Transaction log backups
Monday	-	00:00, 06:00, 12:00, 18:00	Every 5 minutes, 24/7
Tuesday	-	00:00, 06:00, 12:00, 18:00	Every 5 minutes, 24/7
Wednesday	-	00:00, 06:00, 12:00, 18:00	Every 5 minutes, 24/7
Thursday	-	00:00, 06:00, 12:00, 18:00	Every 5 minutes, 24/7
Friday	-	00:00, 06:00, 12:00, 18:00	Every 5 minutes, 24/7
Saturday	-	00:00, 06:00, 12:00, 18:00	Every 5 minutes, 24/7
Sunday	00:00	06:00, 12:00, 18:00	Every 5 minutes, 24/7

The full backup provides the base for recovery. Differential backups reduce the number of files that must be restored, and log backups capture recent transactions. Backup times should be adjusted if monitoring shows that jobs overlap or affect website performance.

3. Backup locations and ransomware protection

The plan follows the 3-2-1-1-0 rule: keep three copies of the data, use at least two types of storage, keep one copy off-site, keep one copy offline or immutable, and test backups so there are no undetected errors. The copies will be the production database, a backup on a separate

local NAS or backup server, and a copy in off-site cloud storage. At least one backup copy will be immutable or kept offline. Backups will be encrypted, access will be restricted, and backup credentials will be separate from production credentials.

4. Automation

Scheduled jobs will create full, differential, and transaction log backups. Monitoring will record each job's result and send an alert if a backup fails. Copies will also be transferred to off-site and immutable storage, and restoration tests will be performed regularly. SQL Server Agent can be used if it is installed and running; otherwise, scheduled scripts can run the backup commands.

5. Restore after a failure on Wednesday at 15:00

First, if the log file of the damaged database is still readable, we take a tail-log backup to save the last transactions. Then we restore the full backup of Sunday 00:00, using NORECOVERY. Next, we restore Wednesday's 12:00 differential backup, also using NORECOVERY. Then we restore each transaction log backup created after that differential, in chronological order, using NORECOVERY. The last available log backup is restored with RECOVERY to bring the database online. With log backups every five minutes, the expected data loss should be within the 15-minute RPO, provided the backup files are available and the log chain is intact. The actual recovery time must be confirmed through tests to ensure it stays under one hour.